

Phase 1 Specification (KT1 — First Checkpoint): Movie NLP Success Analyzer

Course: Systems for Research and Data Analysis (SIAP)

Team: Laslo Uri (E2 163/2023), Denis Dautović (E2 166/2023), Nikola Janković (E2 130/2025)

Date: March 2026

Checkpoint: KT1 (First checkpoint)

Table of Contents

- [Abstract](#)
- [1. Introduction and project goal](#)
- [2. Data collection and preparation](#)
 - [2.1 Pipeline overview](#)
 - [2.2 Step by step — what each script does](#)
 - [2.3 What we have in the data at the end](#)
- [3. What we have in Phase 1 \(Phase 01\) — overview](#)
- [4. Notebook 01 — Exploratory data analysis \(EDA\)](#)
 - [4.1 Notebook structure and what each section does](#)
- [5. Notebook 02 — NLP feature extraction](#)
 - [5.1 What each section does](#)
- [6. Notebook 03 — Baseline models](#)
 - [6.1 What each section does](#)
 - [6.2 What the baseline model figures show](#)
- [7. What we use, why, what we expect and what we got](#)
 - [7.1 Techniques and tools](#)
 - [7.2 Expectations \(KT1\) and achieved results](#)
 - [7.3 Are we satisfied?](#)
- [8. Conclusion](#)

Abstract

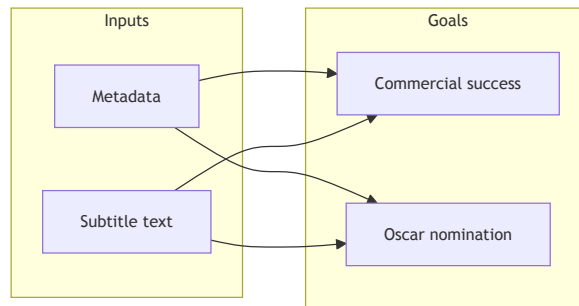
This document is the **specification for the first phase (first checkpoint — KT1)** of the *Movie NLP Success Analyzer* project. It describes the full pipeline within KT1: from collecting data from multiple sources (TMDB, subtitles, awards) through exploratory analysis, extracting NLP features from subtitle text, and training baseline models for two binary predictions — commercial success ($\text{ROI} \geq 2$) and Oscar nomination. The specification aims to define clearly and understandably what we have, how we obtained it, what each step does (including notebook cells and the meaning of each figure), which techniques we use and why, and whether we are satisfied with the results. The document is intended for readers without deep prior knowledge of algorithms and NLP techniques.

1. Introduction and project goal

The project goal is to **predict movie success** based on:

1. **Commercial success** — whether the movie will achieve revenue at least twice the budget ($ROI \geq 2$).
2. **Critical success** — whether the movie will receive an Oscar nomination.

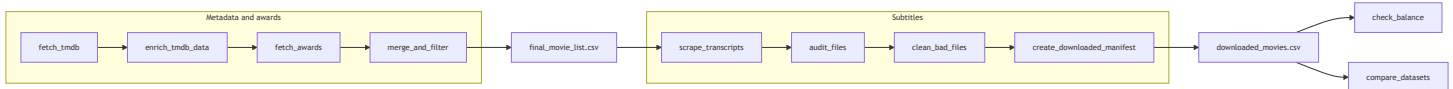
For this we use both **metadata** (budget, runtime, genre, vote count) and **subtitle text** (dialogue), from which we extract natural language processing (NLP) features: sentiment, lexical diversity, and TF-IDF representation of the text. Phase 1 establishes a **reproducible baseline**: a clean dataset, EDA, NLP features, and three baseline classifiers (logistic regression, random forest, XGBoost) without hyperparameter optimization.



2. Data collection and preparation

Data were collected in **one sequential pipeline** of ten scripts in the `data_pipeline/` module. Each script has a single role; the output of one serves as the input to the next. At the end we obtain one main table for modeling and a folder with subtitle texts for NLP.

2.1 Pipeline overview



Why this order? First we build the movie list with financial and award data; only then do we download subtitles for those movies and check file quality. Finally we check class balance and any scraper bias.

2.2 Step by step — what each script does

Script, input and output	Description
1. fetch_tmdb.py Input: TMDB API key (<code>.env</code>). Output: <code>data/raw/tmdb_metadata/movies_YYYY.csv</code> (76 files).	Fetches movie metadata from the TMDB API: by year from 1950 to 2025, 500 “top” movies per year. Saves results to CSV by year so nothing is lost if the script fails.
2. enrich_tmdb_data.py Input: Raw TMDB CSVs. Output: <code>data/processed/tmdb_enriched.csv</code> .	For each movie from the raw files, fetches budget , revenue , and IMDb ID (needed for later subtitle download). Processes in batches and pauses if the API rate-limits.
3. fetch_awards.py Input: Wikipedia API. Output: <code>awards_master_partial.csv</code> , <code>awards_minor_categories.csv</code> .	Collects film award data (Oscar, BAFTA, Golden Globe, etc.) via the Wikipedia API. Parses HTML tables, cleans movie titles, and builds a unique list of awarded/nominated movies.
4. merge_and_filter.py Input: TMDB + enriched + awards. Output:	“Big filter” : merges TMDB metadata, enriched finances, and awards into one master table. Keeps only movies that satisfy

Script, input and output	Description
data/processed/final_movie_list.csv (12,722 movies).	at least one condition: valid financial data (budget/revenue > 1000), major award, or enough votes (evidence of relevance). Also computes labels: <code>is_commercial</code> ($\text{ROI} \geq 2$), <code>is_award_winner</code> .
5. scrape_transcripts.py Input: <code>final_movie_list.csv</code> . Output: <code>data/raw/subtitles/{tmdb_id}.txt</code> (~7,400 files).	Downloads subtitle text from the web for movies on the final list. Uses User-Agent header rotation to avoid blocking; fuzzy title matching ensures the correct subtitle is downloaded.
6. audit_files.py Input: folder <code>subtitles/</code> . Output: <code>data/processed/audit_report.csv</code> .	Checks quality of downloaded files: flags files that are too small (< 2 KB), contain error HTML (e.g. 404), or Cloudflare blocks.
7. clean_bad_files.py Input: Audit report. Output: corrupted files removed.	Deletes corrupted files from the audit report. Asks for manual confirmation (“DELETE”) in the terminal before deleting.
8. create_downloaded_manifest.py Input: <code>subtitles/</code> , <code>final_movie_list.csv</code> . Output: <code>data/processed/downloaded_movies.csv</code> .	Compares what physically exists on disk with the final movie list and creates the final manifest : a CSV where each row is guaranteed to have a corresponding valid <code>.txt</code> file.
9. check_balance.py Input: <code>downloaded_movies.csv</code> . Output: console report.	Analyzes class balance (e.g. how many award winners vs. others). Useful to see in advance whether SMOTE or other balancing will be needed.
10. compare_datasets.py Input: final list vs. downloaded. Output: comparison plots in <code>data/processed/</code> .	Visually checks whether scraping introduced systematic bias (e.g. whether older movies or certain genres are under-represented in downloaded subtitles). Saves plots to <code>data/processed/</code> .

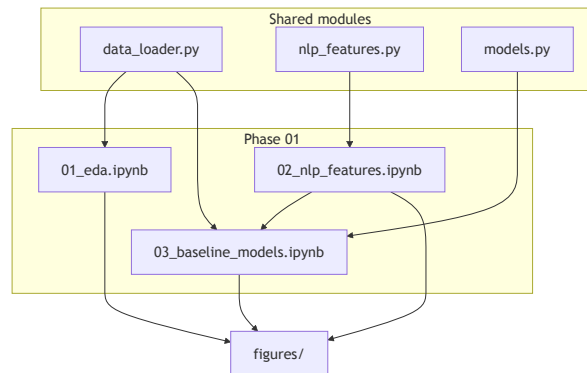
2.3 What we have in the data at the end

- **final_movie_list.csv** — 12,722 movies, 15 columns: `id` , `budget` , `revenue` , `imdb_id` , `runtime` , `release_date` , `genres` , `original_title` , `title` , `vote_count` , `is_commercial` , `is_award_winner` , `is_popular` , `genres_list` , `roi` . This is the main source for EDA and for joining with NLP features.
- **data/raw/subtitles/** — about 7,423 text files (`{tmdb_id}.txt`), one per movie. The text is raw subtitle content (with timestamps, tags, etc.), which in Phase 1 is cleaned and used for NLP feature extraction.

Why it matters: Without this pipeline we would have neither a single table with targets (`is_commercial` , `is_award_winner`) nor text for NLP. We avoided data leakage by **not** including **revenue** in the features for commercial success prediction (since `is_commercial` is defined as $\text{ROI} = \text{revenue}/\text{budget} \geq 2$).

3. What we have in Phase 1 (Phase 01) — overview

Phase 1 consists of three Jupyter notebooks, shared code in `src/`, and a figures folder.



Component	Purpose
01_eda.ipynb	Exploratory analysis: dataset structure, target distributions, finances, genres, time, missing data, subtitle coverage, correlations. Produces 16 figures in <code>figures/</code> .
02_nlp_features.ipynb	Loads subtitles, cleans text, extracts NLP features (text statistics, lexical diversity, VADER sentiment, TF-IDF) and saves <code>nlp_features.csv</code> and the TF-IDF matrix. Produces 6 figures.
03_baseline_models.ipynb	Loads main dataset and NLP features, defines 8 progressive feature sets, trains LR / RF / XGB for both targets (commercial and award), shows tables and heatmaps. Produces 10 figures.
src/data_loader.py	<code>load_main_dataset</code> , <code>load_nlp_features</code> , <code>merge_datasets</code> , <code>get_unique_genres</code> , <code>create_genre_dummies</code> — loading and merging data for EDA and modeling.
src/nlp_features.py	Text cleaning, basic statistics, lexical diversity, sentiment (VADER), TF-IDF; batch processing.
src/models.py	<code>prepare_modeling_data</code> , <code>create_splits</code> , <code>get_baseline_models</code> , <code>train_baseline</code> , functions for result tables, confusion matrices, ROC curves, feature importance.

4. Notebook 01 — Exploratory data analysis (EDA)

Notebook **01_eda.ipynb** uses the main dataset (`final_movie_list.csv`) loaded via `src.data_loader.load_main_dataset` . The loader parses the `genres` column into a list and adds the `roi` column (revenue/budget). The goal of EDA is to understand distributions, missing values, correlations, and data readiness for modeling.

4.1 Notebook structure and what each section does

- **Introduction and setup**

Imports: `pandas` , `numpy` , `matplotlib` , `seaborn` , and `load_main_dataset` , `safe_literal_eval` . Paths set: `DATA_DIR` , `PROCESSED_DIR` , `RAW_DIR` , `SUBTITLES_DIR` , `FIGURES_DIR` .

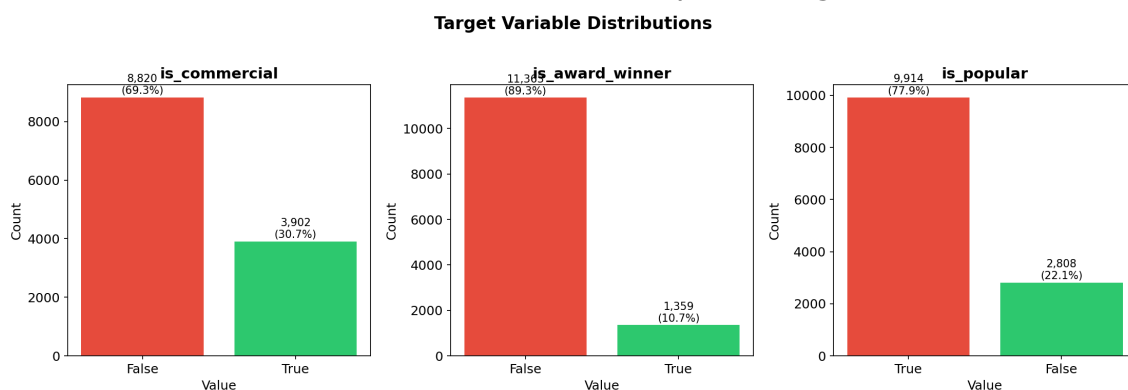
- **1. Dataset overview**

- Loads the dataset and prints `shape` (12,722 × 15) and column list.
- Explanation: target variables are `is_commercial` , `is_award_winner` , `is_popular` .
- `df.dtypes` — column types.
- `df.describe()` — basic statistics for numeric columns (including ROI).
- First rows — what one record looks like (budget, revenue, genres, labels).

- **2. Target distributions**

Visualization of movie counts by target variable (commercial success, award, popularity). Shows imbalance: e.g. awards are a minority class (~13%).

Figure: `01_eda_01_target_distributions.png` — bar plot of target distributions.

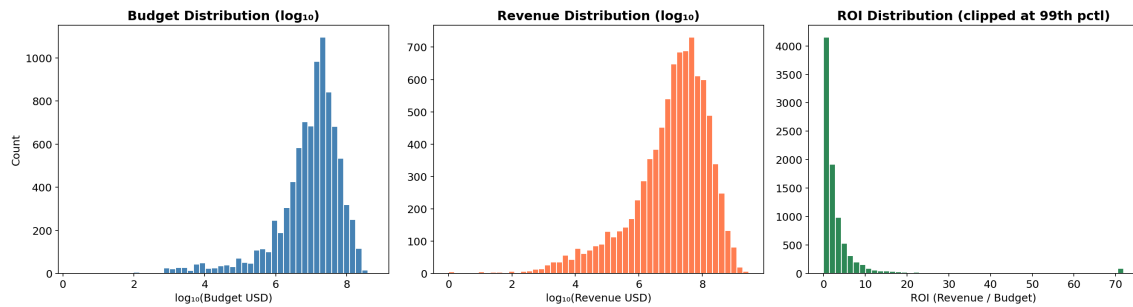


What the figure shows: Number of movies with True/False for each of the three targets. Commercial success and popularity are roughly balanced; awards are a minority class (~13% True), which calls for SMOTE or `class_weight` when training.

- **3. Financial distributions**

Histograms for `budget` , `revenue` , `roi` (often on log scale due to extreme values).

Figure: 01_eda_02_financial_distributions.png — distributions of budget, revenue, and ROI.

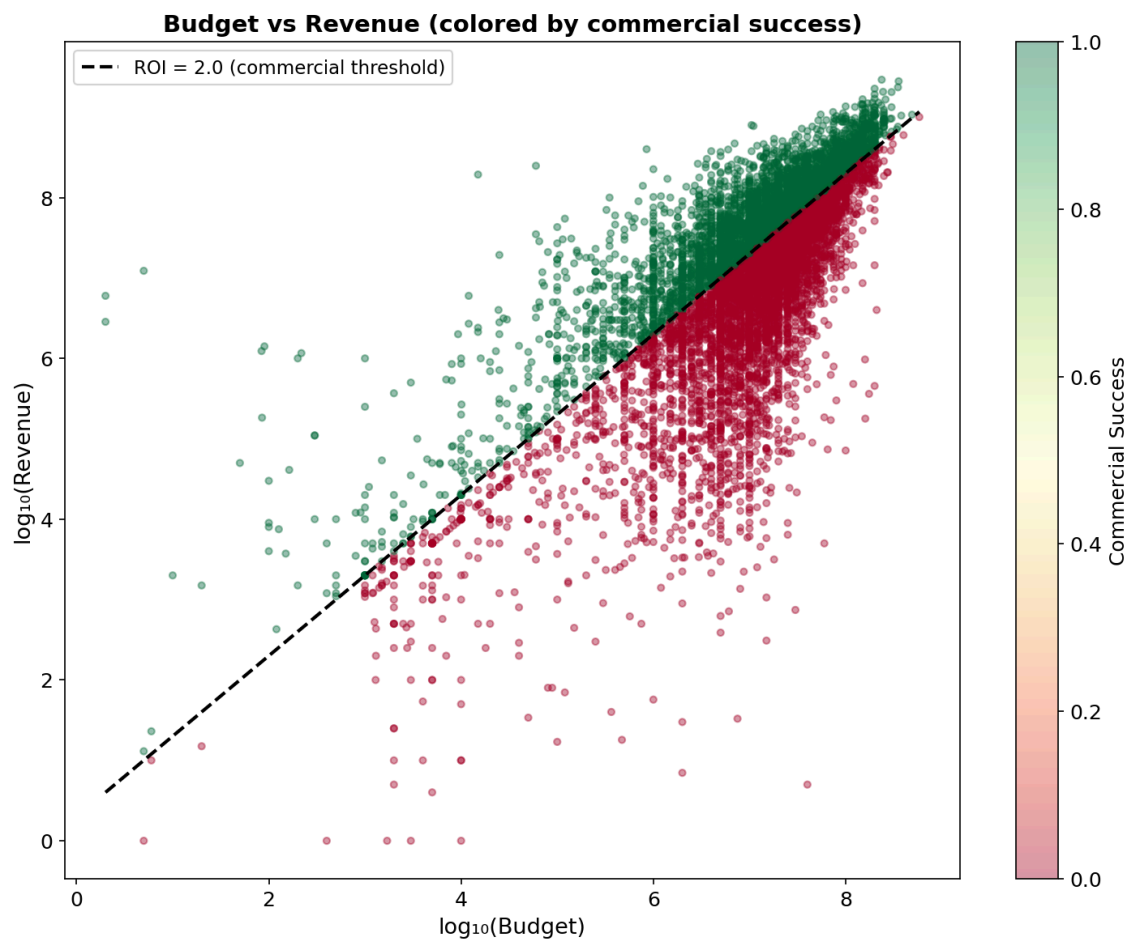


What the figure shows: Histograms are strongly right-skewed — most movies have low budget/revenue, few have very high values. That is why we use log transformation in the model (e.g. budget_log).

- **4. Budget vs revenue**

Scatter plot of budget vs. revenue (usually log-log). Shows that higher budget does not guarantee higher revenue; there are outliers.

Figure: 01_eda_03_budget_vs_revenue.png .

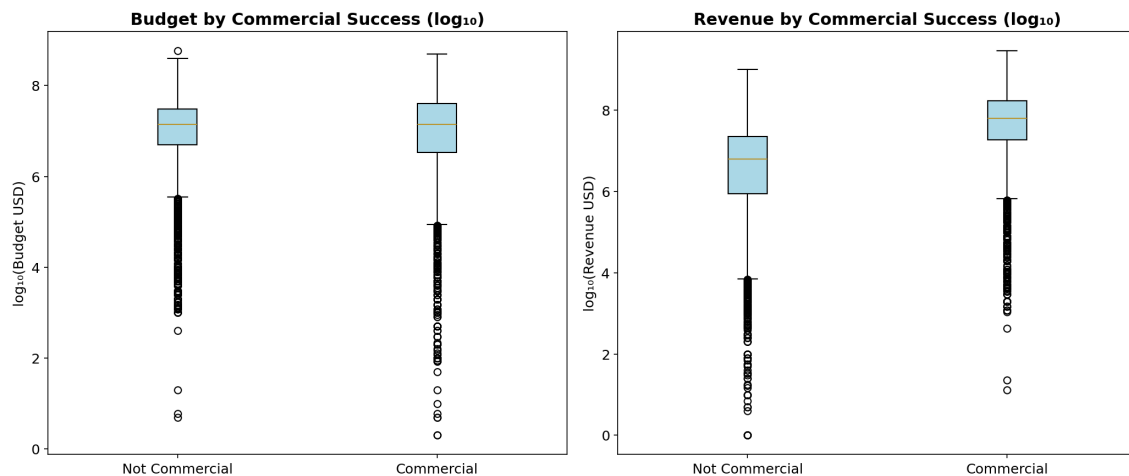


What the figure shows: Spread in log-log scale: higher budget does not guarantee higher revenue. There are clear outliers (movies with low budget and high revenue). This confirms that predicting commercial success is not trivial.

- **5. Financial boxplots by success**

Boxplots of financial metrics split by `is_commercial` (and possibly `is_award_winner`). Successful movies have higher median revenue.

Figure: 01_eda_04_financial_boxplots.png .

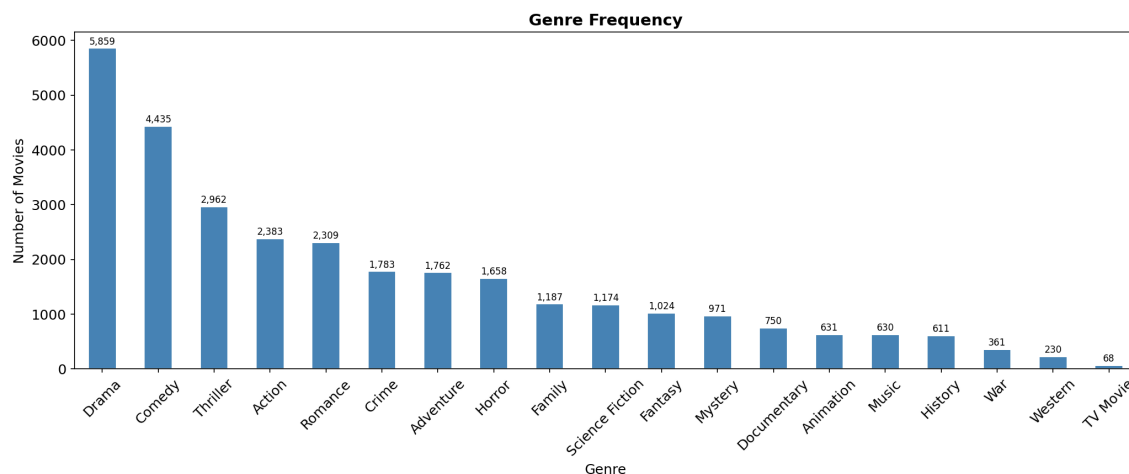


What the figure shows: When we split movies into successful and unsuccessful (by `is_commercial` or `is_award_winner`), successful ones have higher median revenue; the difference in budget is less pronounced. Financial data carries signal for the target.

- **6. Genre frequency**

Number of movies per genre (from `genres_list`). E.g. Drama, Comedy, Thriller are among the most common.

Figure: 01_eda_05_genre_frequency.png .

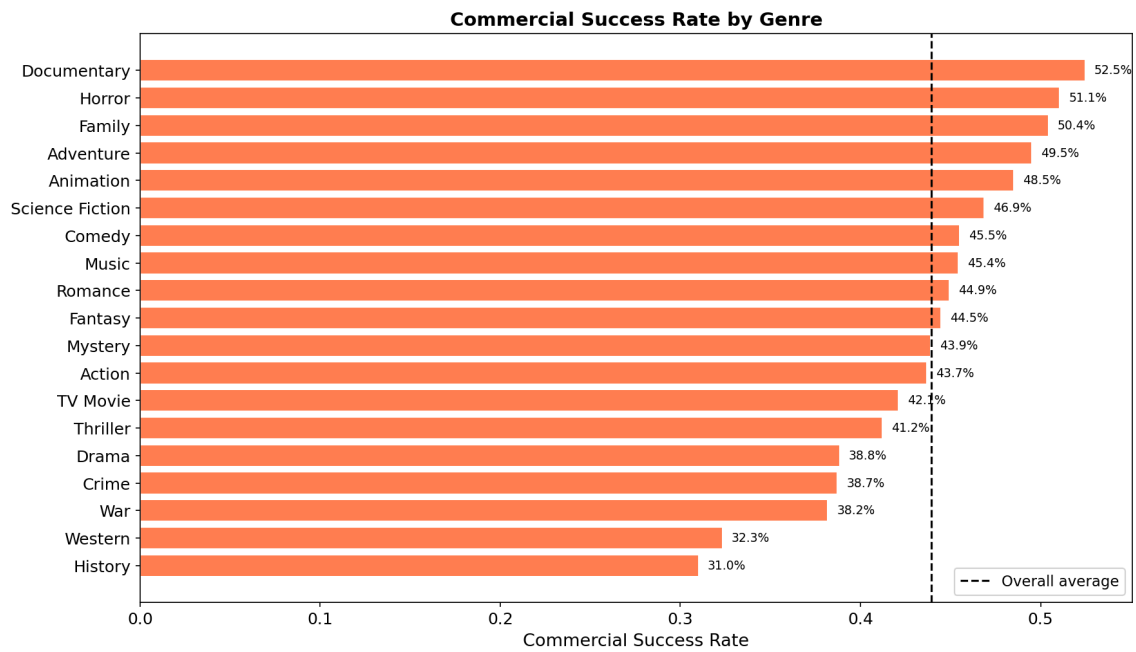


What the figure shows: Drama, Comedy, and Thriller dominate by count; genre is important for the model because we encode it as dummy features (e.g. `genre_drama`).

- **7. Genre success rates**

Success rate (e.g. share of `is_commercial == True`) per genre. E.g. Adventure, Sci-Fi have more successes; Documentary, TV Movie fewer.

Figure: 01_eda_06_genre_success_rates.png .

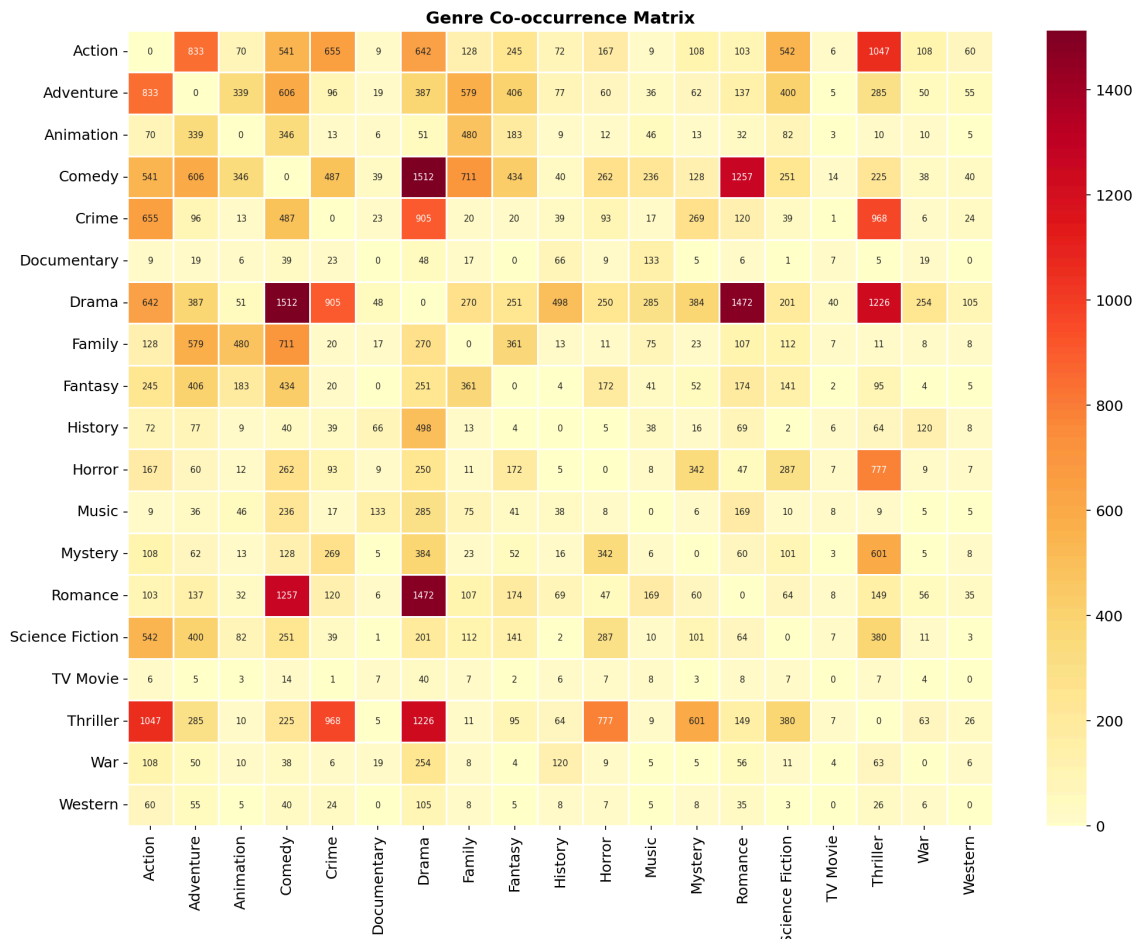


What the figure shows: Success rates ($ROI \geq 2$ or award) differ by genre: Adventure, Sci-Fi, Action often have more successes; Documentary, TV Movie fewer. Genre is a useful feature for both targets.

• 8. Genre co-occurrence

Which genres often appear together (e.g. Drama + Romance).

Figure: 01_eda_07_genre_cooccurrence.png .

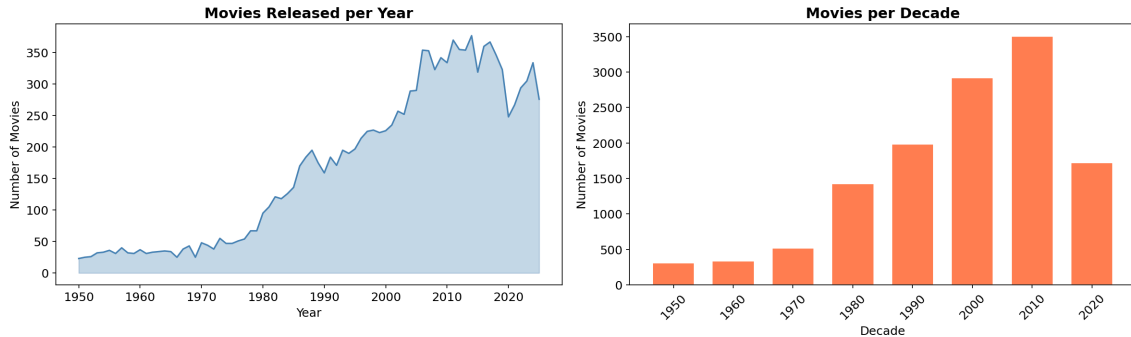


What the figure shows: Which genres co-occur (e.g. Drama + Romance). Movies have multiple genres; the heatmap helps see data structure and avoid duplicating signal in dummy features.

- **9. Temporal production**

Number of movies per year or decade — growth of production over time.

Figure: 01_eda_08_temporal_production.png .

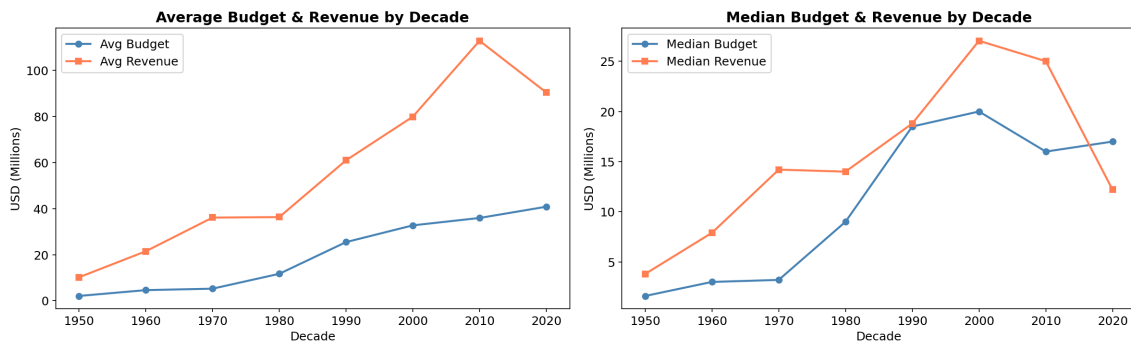


What the figure shows: Number of movies per year/decade grows sharply — from a few hundred in the 1950s to thousands in the 2010s. The dataset is not uniform over time; release_year can be a useful feature.

- **10. Temporal financials**

Average budget/revenue by time period.

Figure: 01_eda_09_temporal_financials.png .

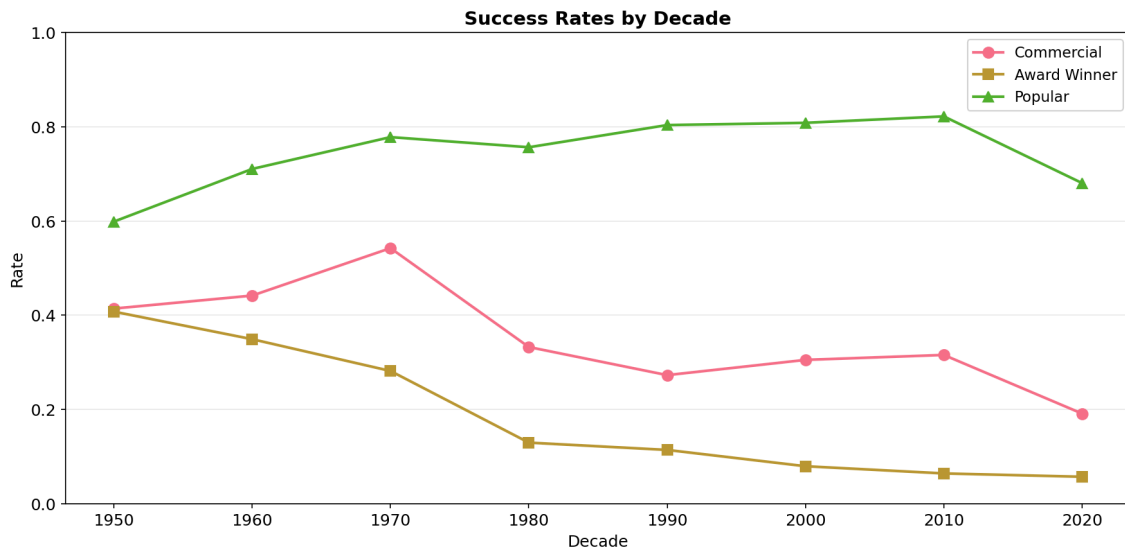


What the figure shows: Average budget and revenue by decade — they grow over time, peaking in the 2000s/2010s. Confirms that the time component can contribute to the model.

- **11. Temporal success rates**

Commercial success rate and award rates by decade. Shows a drop in award rate in recent decades (more movies, same number of awards).

Figure: 01_eda_10_temporal_success_rates.png .

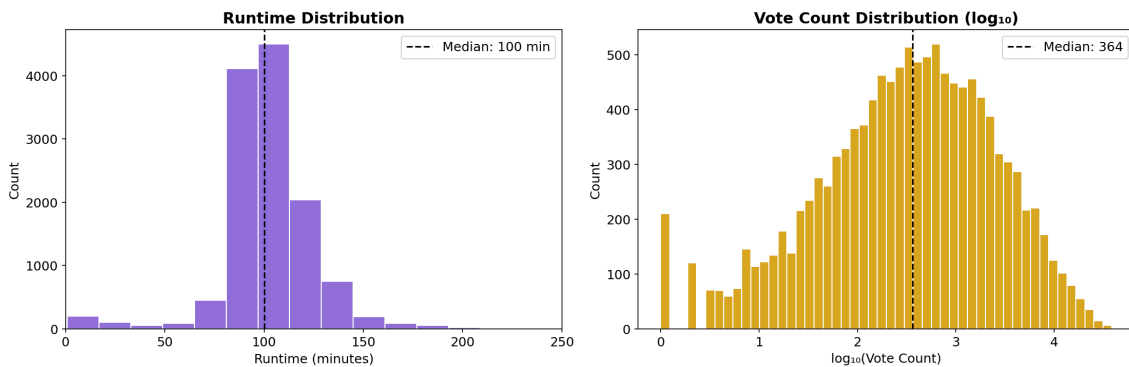


What the figure shows: Commercial success rate and share of awarded movies by decade. In recent decades the share of awarded movies drops (more movies, same number of awards) — important for interpreting imbalance.

• 12. Runtime and vote count distributions

Histograms for runtime and vote_count.

Figure: 01_eda_11_runtime_votes_distributions.png.

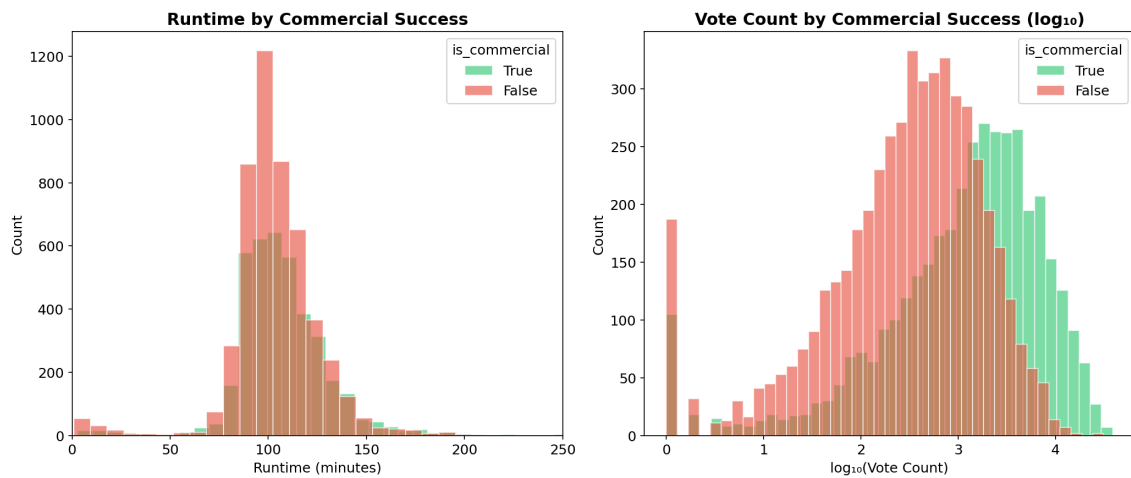


What the figure shows: Runtime is clustered around 90–120 min; vote_count is strongly right-skewed (few movies have thousands of votes). That is why we use vote_count_log in the model.

• 13. Runtime and votes by success

Boxplots of runtime/vote_count split by success (commercial / award).

Figure: 01_eda_12_runtime_votes_by_success.png.

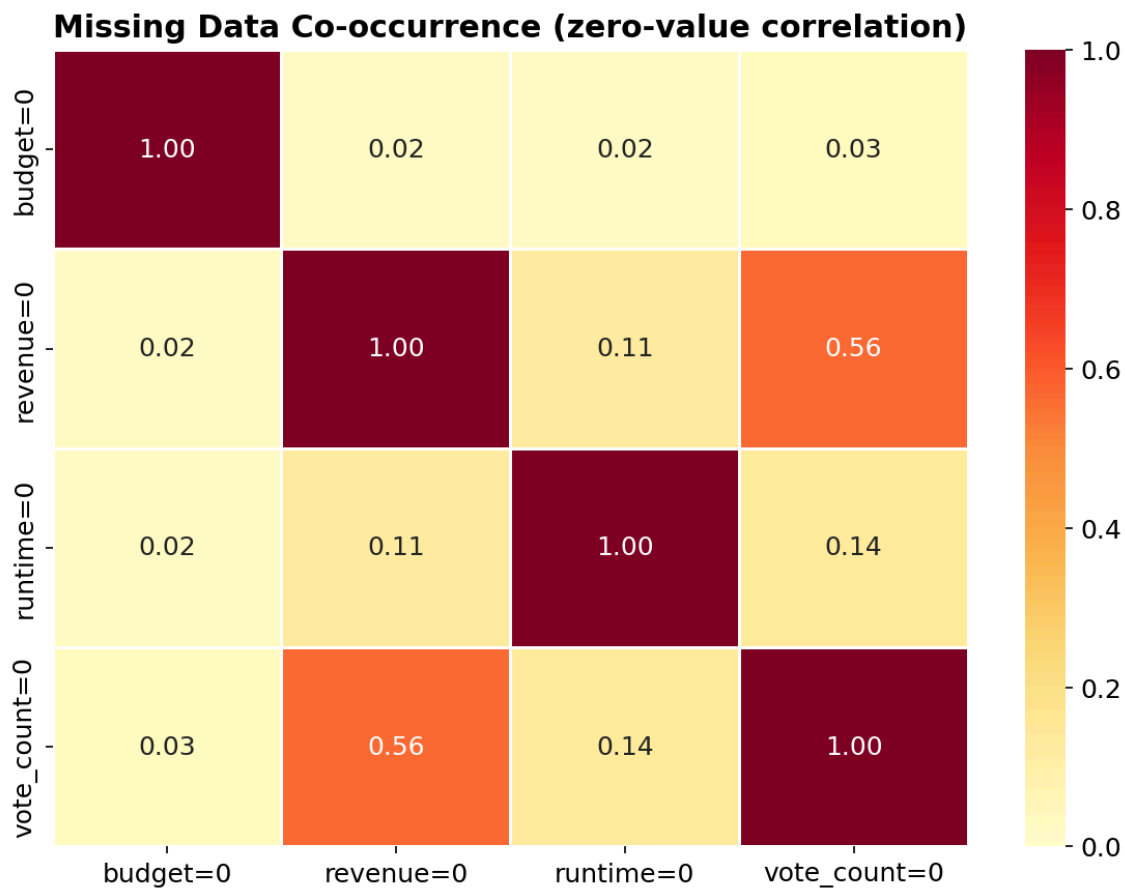


What the figure shows: Successful movies (commercial or awarded) have on average more votes and slightly longer runtime. Both features are justified in the progressive sets.

• 14. Missing data heatmap

Heatmap of missing values by column (or by sample). Shows where gaps are (e.g. budget = 0, revenue = 0).

Figure: 01_eda_13_missing_data_heatmap.png .

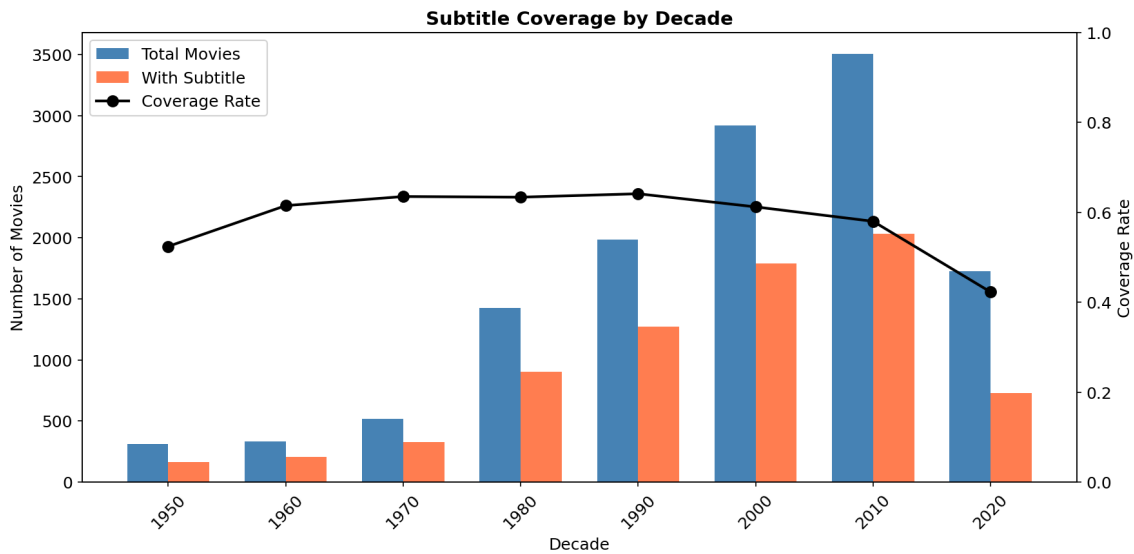


What the figure shows: Where data are missing (e.g. budget = 0, revenue = 0). For modeling we filter to movies with valid budget, revenue, and runtime so we do not introduce fake values.

• **15. Subtitle coverage**

How many movies have subtitles (overlap with `data/raw/subtitles/`). Usually a bar or pie plot: e.g. ~58% have subtitles.

Figure: `01_eda_14_subtitle_coverage.png` .

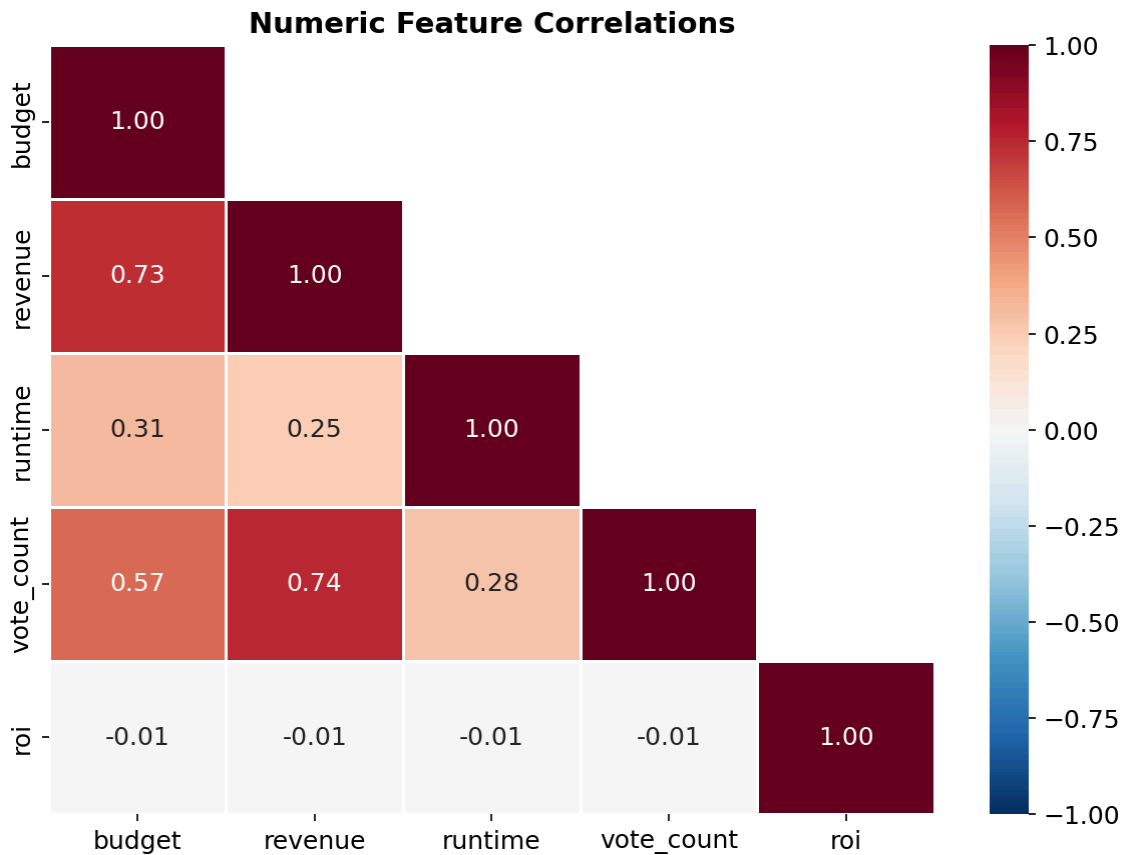


What the figure shows: About 58% of movies have subtitles; the rest are not in the modeling subset with NLP features. Coverage is sufficient for meaningful text analysis.

• **16. Feature correlations**

Correlation matrix of numeric features (Pearson).

Figure: `01_eda_15_feature_correlations.png` .

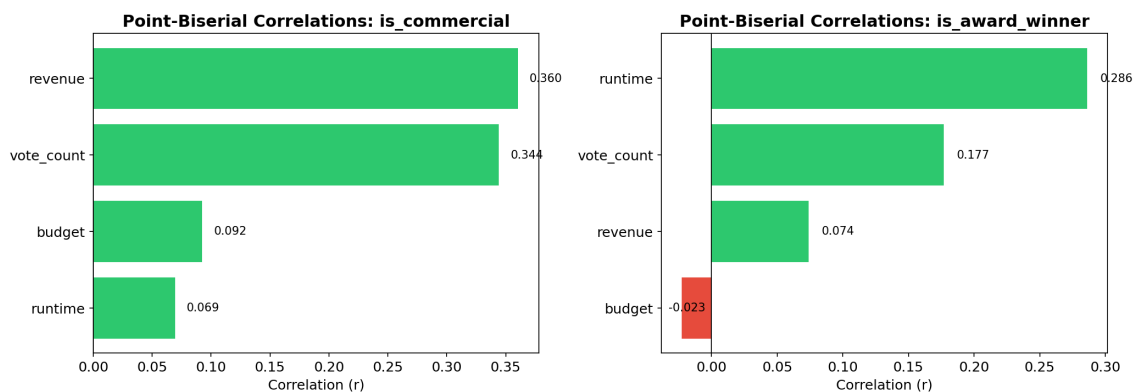


What the figure shows: Pearson correlations between numeric features. Budget, revenue, and vote_count are correlated; useful for understanding multicollinearity and why we omit revenue for commercial.

- **17. Point-biserial correlations**

Correlation of each numeric feature with the binary target (`is_commercial` , `is_award_winner`). Shows what is most associated with success: e.g. `vote_count` , `revenue` (for commercial, but revenue must not be used due to leakage), `runtime` for awards.

Figure: 01_eda_16_point_biserial_correlations.png .



What the figure shows: Strength of association of each feature with the target. For commercial success the strongest are `vote_count` and `revenue` (we must not use revenue); for awards, `runtime` and `vote_count`. This guides feature choice in sets 7 and 8.

EDA conclusion: The dataset is suitable for modeling; there are clear correlations with targets; imbalance for awards requires attention (SMOTE or `class_weight`); subtitle coverage (~58%) is sufficient for NLP.

5. Notebook 02 — NLP feature extraction

Notebook **02_nlp_features.ipynb** loads the movie list and the subtitles folder, cleans text, and calls functions from `src/nlp_features.py` to compute all features. Results are saved to `nlp_features.csv` and the TF-IDF matrix (+ vocabulary and corpus IDs) for use in notebook 03.

5.1 What each section does

- **Setup**

Imports modules from `nlp_features` : text cleaning, basic statistics, lexical diversity, sentiment (VADER), punctuation, sentiment variance, readability, TextBlob, batch processing, TF-IDF.

- **1. Subtitle loading**

Loads `final_movie_list.csv` , scans `data/raw/subtitles/` , and maps `tmdb_id` → file path. Computes how many movies have subtitles (e.g. 7,423 of 12,722). Shows a short example of subtitle content (raw text with timestamps).

- **2. Text cleaning demo**

Two functions: **strict cleaning** (for statistics and TF-IDF) — removes timestamps, HTML tags, brackets, non-ASCII; **cleaning for sentiment** — gentler, keeping punctuation important for emotion. Before/after example on a short text.

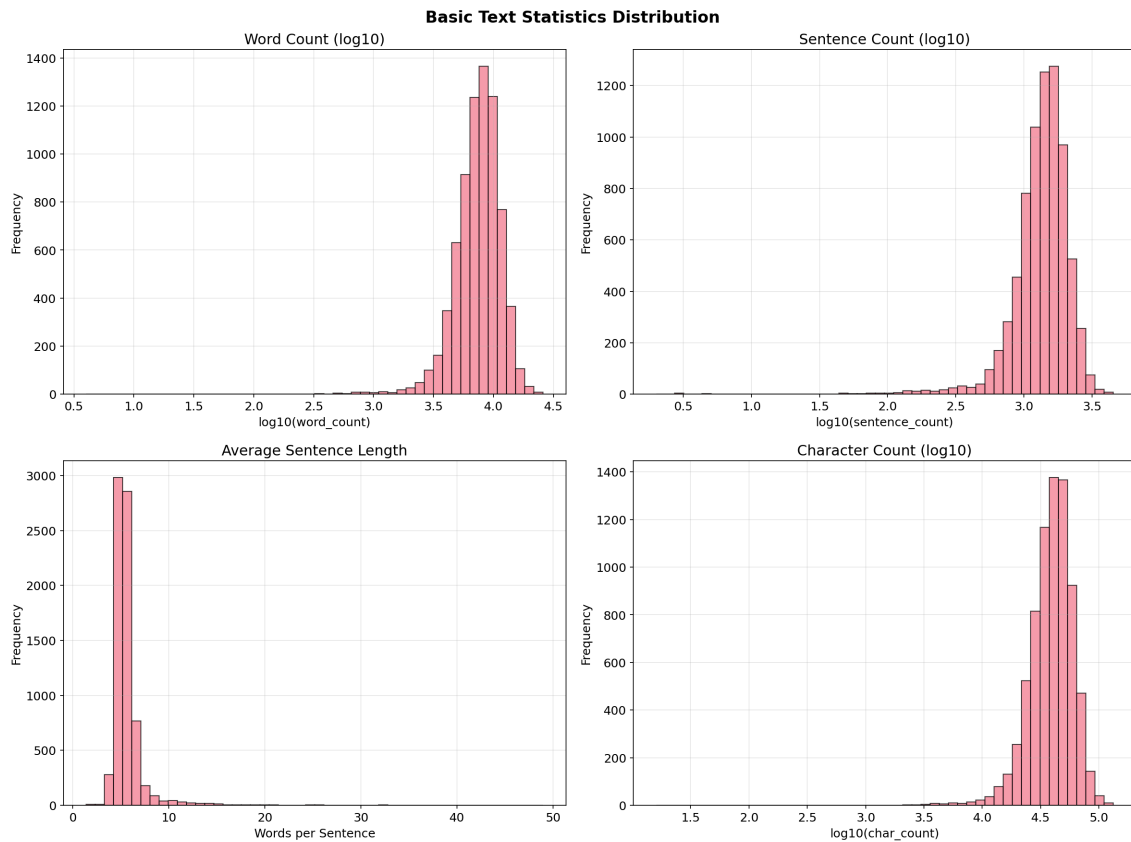
- **3. Full feature extraction**

Calls `process_subtitle_batch()` for all files: for each movie we extract: word count, sentence count, average sentence length, character count, type-token ratio, hapax legomena ratio, unique words; VADER sentiment (compound, pos, neg, neu); exclamation_ratio, question_ratio, sentiment_compound_std, emotional_intensity; avg_word_length, sentence_length_std; TextBlob polarity and subjectivity. Result is a table with ~22 columns (including `id`). Duration: a few minutes for ~7,400 files.

- **4. Basic text stats — visualization**

Histograms or boxplots for `word_count` , `sentence_count` , `avg_sentence_length` , `char_count` .

Figure: `02_nlp_features_01_nlp_basic_text_stats.png` .

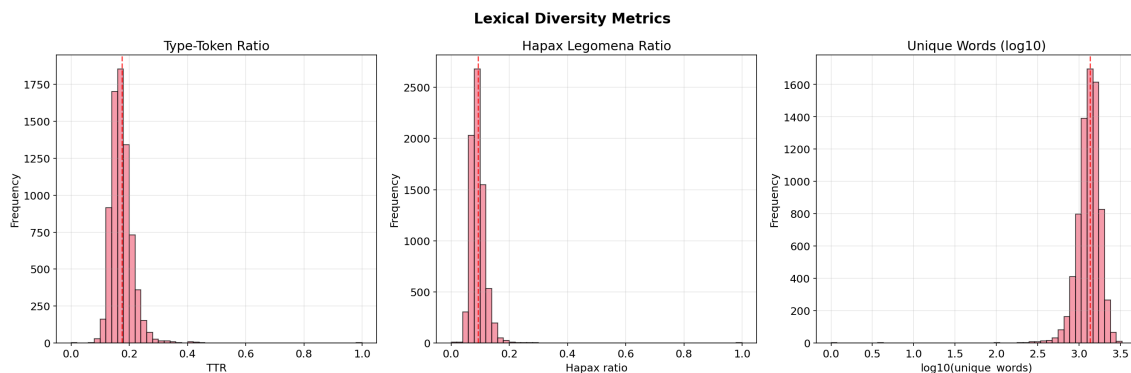


What the figure shows: Distributions of word count, sentence count, average sentence length, and character count per movie. Subtitles are typically thousands of words; these measures go into set 1_text_basic.

• 5. Lexical diversity

Visualization of type-token ratio, hapax legomena ratio, unique_words.

Figure: 02_nlp_features_02_nlp_lexical_diversity.png .

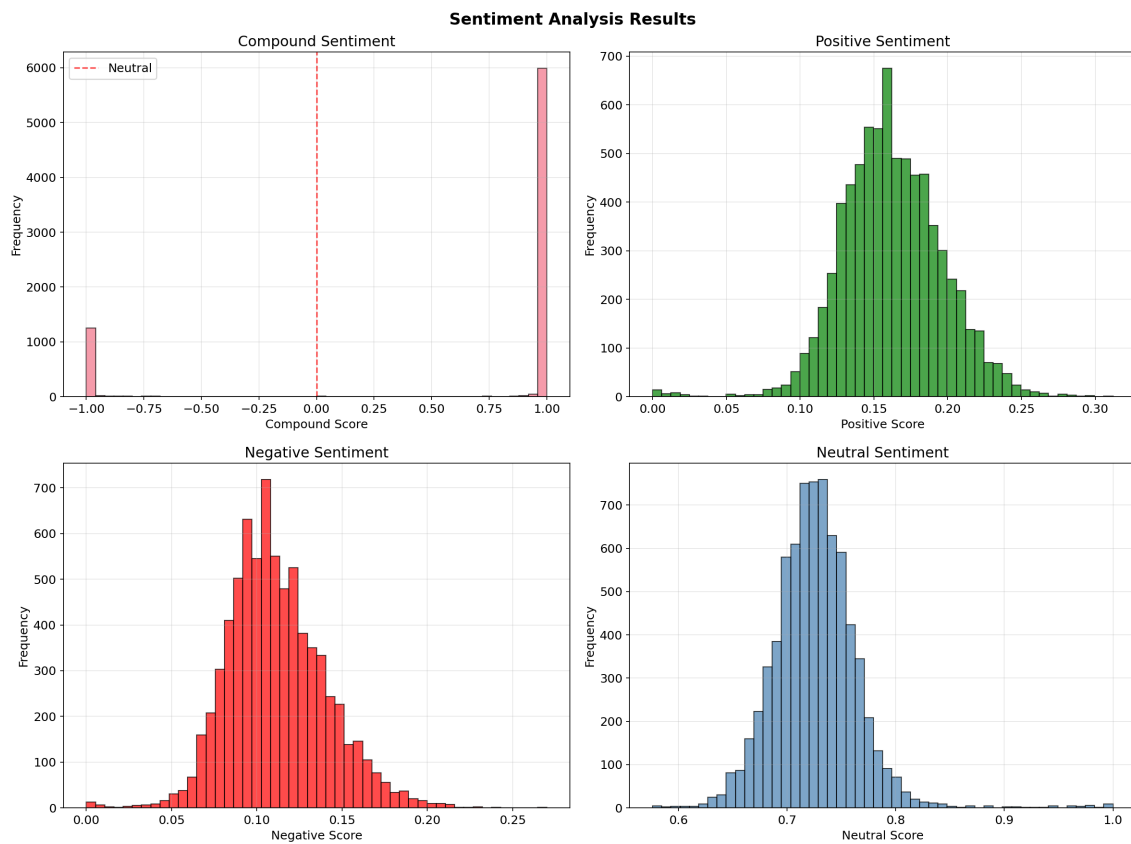


What the figure shows: Type-token ratio, hapax legomena ratio, and number of unique words — measures of vocabulary richness. Lower values mean more repetitive dialogue; used in the text_basic set.

• 6. Sentiment histograms

Distributions of VADER compound, pos, neg, neu. Most movies have positive compound (dialogue is often positive overall).

Figure: 02_nlp_features_03_nlp_sentiment_histograms.png .

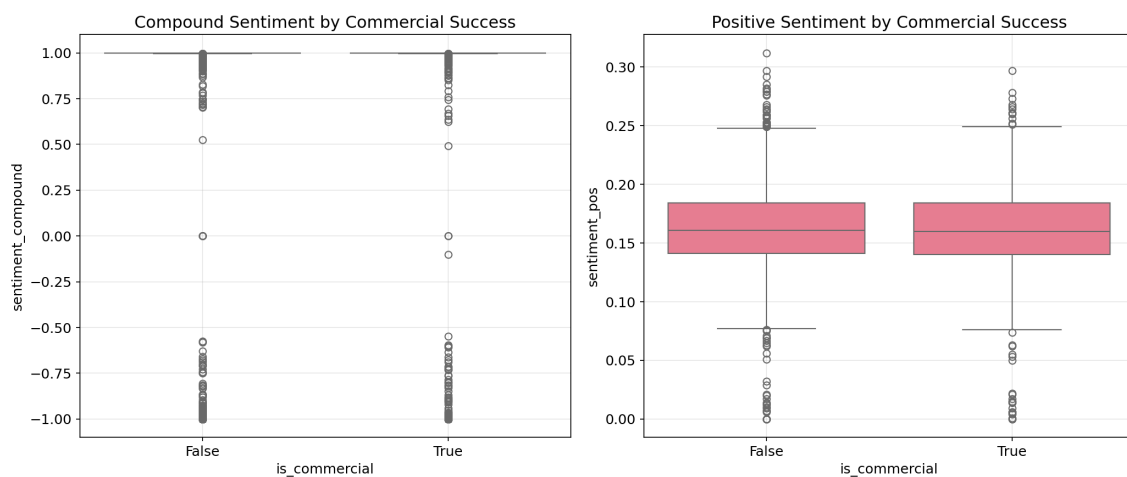


What the figure shows: VADER sentiment (compound, positive, negative, neutral) per movie. Most subtitles have positive compound — dialogue is overall positive; the neutral component dominates. These columns go into set 2_text_vader.

• 7. Sentiment by success

Sentiment (e.g. compound) split by `is_commercial` or `is_award_winner`. Whether successful movies have different sentiment.

Figure: 02_nlp_features_04_nlp_sentiment_by_success.png .

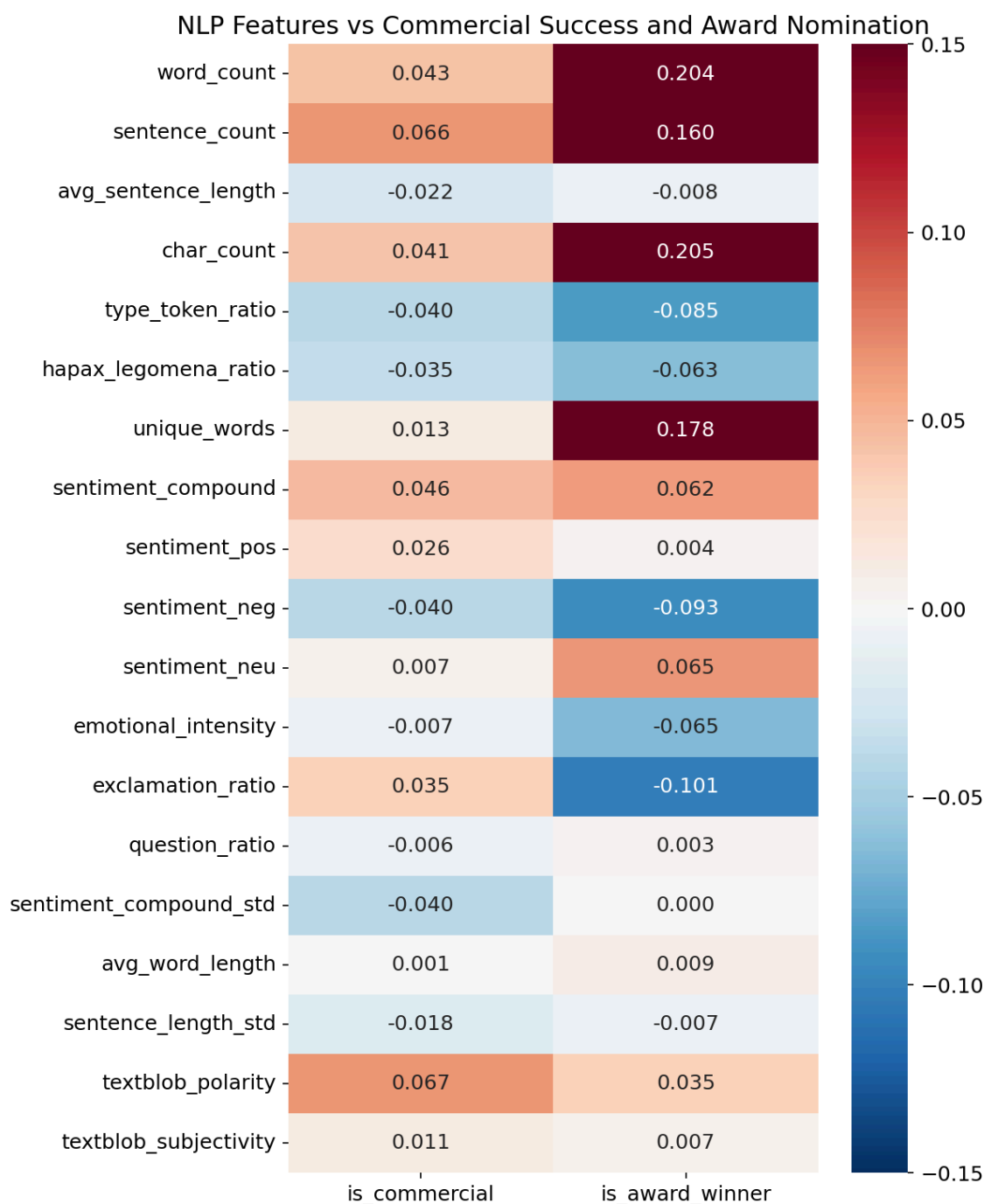


What the figure shows: Comparison of sentiment (e.g. compound) between successful and unsuccessful movies. If distributions differ, sentiment carries signal for the model; otherwise its contribution is limited.

- **8. NLP correlation heatmap**

Correlation matrix between NLP features.

Figure: 02_nlp_features_05_nlp_correlation_heatmap.png .

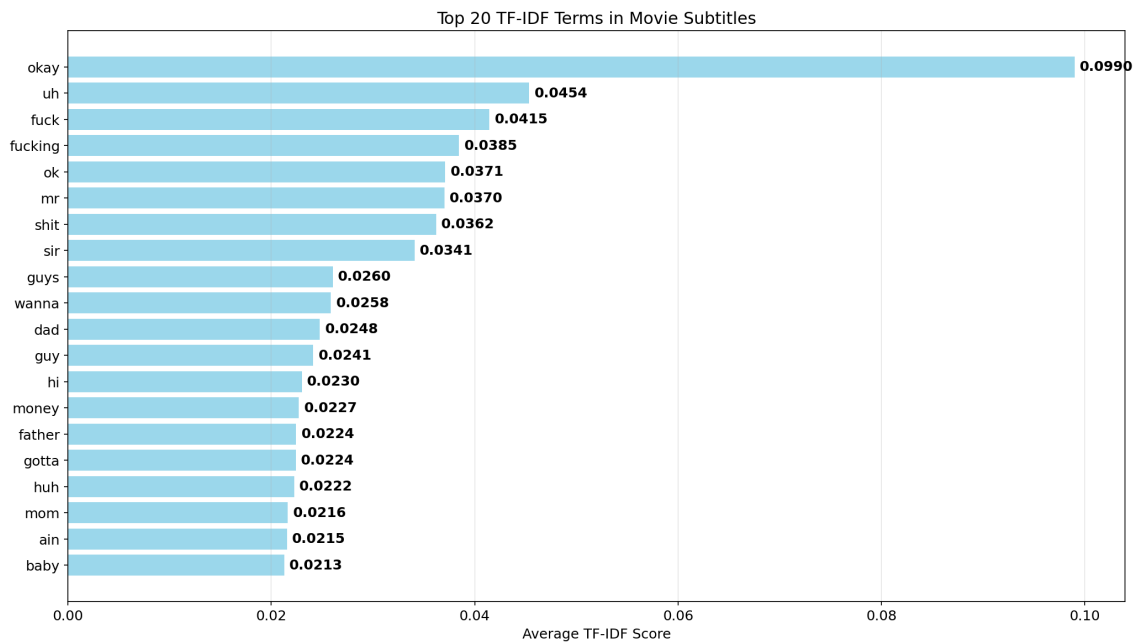


What the figure shows: Correlations between NLP features. Some measures are correlated (e.g. word_count and char_count); useful to avoid duplicating the same signal in the model.

- **9. TF-IDF**

Creates vectorizer (e.g. max 5000 terms, unigrams+bigrams, min_df/max_df, stop words). Fit on cleaned corpus; saves matrix (npz), vocabulary (pkl), and corpus IDs (pkl). Shows top TF-IDF terms (e.g. “okay”, “uh”, “fuck”, “sir” — typical dialogue).

Figure: 02_nlp_features_06_nlp_tfidf_top_terms.png .



What the figure shows: Terms with highest TF-IDF weight in the subtitle corpus — typically everyday dialogue words (okay, uh, sir, fuck). SVD of 5000 terms yields 100 components that go into set 3_text_full.

- **Saving**

`nlp_features.csv` is written to `data/processed/` ; TF-IDF matrix and auxiliary files as well.

Conclusion: We have numeric NLP features for 7,423 movies and a sparse TF-IDF matrix for SVD in notebook 03.

6. Notebook 03 — Baseline models

Notebook **03_baseline_models.ipynb** loads the main dataset and NLP features, merges them, creates genre dummies, filters to movies with subtitles and valid budget/revenue/runtime, adds SVD components from the TF-IDF matrix, and defines **8 progressive feature sets**. Then for each set and both targets it does a stratified 70/15/15 split, optional SMOTE on training, trains LR, RF, and XGBoost, and collects F1, F1 at optimal threshold, AUC, and balanced accuracy. Finally it shows tables, heatmaps, confusion matrices, ROC curves, and feature importance.

6.1 What each section does

- **Setup**

Imports: `data_loader` , `models` , `NumPy`, `Pandas`, `matplotlib`, `seaborn`, `TruncatedSVD`, `scipy.sparse`, `pickle`. Seed set to 42, `paths`, `FIGURES_DIR` .

- **1. Data loading & feature engineering**

- `load_main_dataset` , `load_nlp_features` , `merge_datasets` ; `get_unique_genres` , `create_genre_dummies` .
- Filtering: `word_count` not null, `budget > 0` , `revenue > 0` , `runtime > 0` → **modeling_df** (e.g. 5,754 movies).
- Derived columns: `title_length` , `budget_log` , `revenue_log` , `vote_count_log` , `release_month` , `release_year` .
- Load TF-IDF matrix and corpus IDs; `TruncatedSVD` with 100 components; merge SVD features into `modeling_df` (`tfidf_svd_1 ... tfidf_svd_100`). Explanation: 100 SVD components provide the “text” signal for set 3_text_full.

- **2. Feature set definitions**

- **1_text_basic**: only basic NLP (`word_count`, `sentence_count`, `avg_sentence_length`, `char_count`, `type_token_ratio`, `hapax_legomena_ratio`, `unique_words`).
- **2_text_vader**: + `sentiment_compound`, `sentiment_pos`, `sentiment_neg`, `sentiment_neu`.
- **3_text_full**: + optional NLP (`avg_word_length`, `sentence_length_std`, `textblob_sentiment_compound_std`, `exclamation_ratio`, `question_ratio`, `emotional_intensity`) + all 100 `tfidf_svd` columns.
- **4_genre**: + genre dummies.
- **5_title**: + `title_length`.
- **6_budget**: + `budget_log`.
- **7_runtime**: + `runtime`.

- **8_+vote_revenue:** for **commercial** same as 7 (revenue is not added — leakage); for **award** add `vote_count_log` and `revenue_log`.

Prints number of features per set for commercial and award.

- **3. Progressive training (sets 1–3)**

For sets 1_text_basic, 2_text_+vader, 3_text_full: for each target prepare `modeling_data` → X, y; create `splits` → stratified 70/15/15; train `baseline(..., use_smote=True)` for commercial and award. Results stored in `all_results[set_name]` .

- **4. Progressive training (sets 4–6)**

Same for 4_+genre, 5_+title, 6_+budget.

- **5. Progressive training (sets 7–8)**

Same for 7_+runtime and 8_+vote_revenue.

- **6. Summary table**

`results_table(all_results)` — table of Test F1, F1@opt, AUC, Bal.Acc. by feature set and model for both targets.

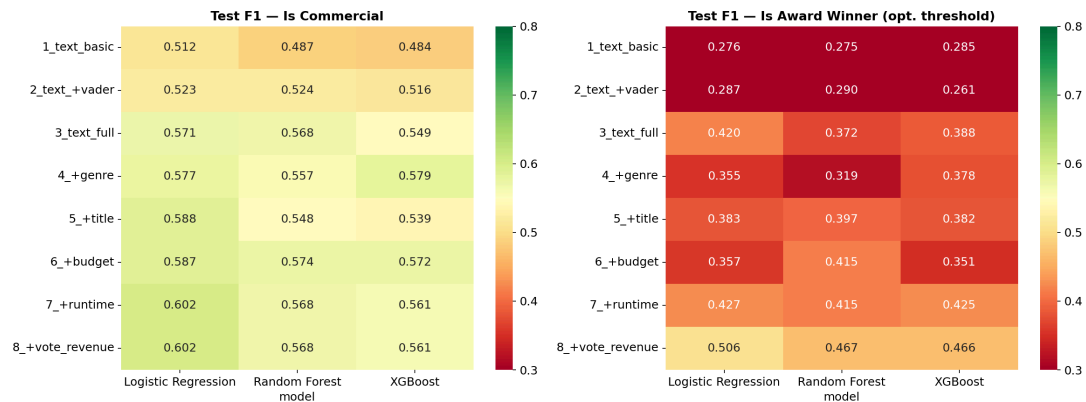
(No separate figure; table is in cell output.)

- **7. Heatmaps**

Heatmaps: F1 by set and model; AUC by set and model; Balanced Accuracy by set and model.

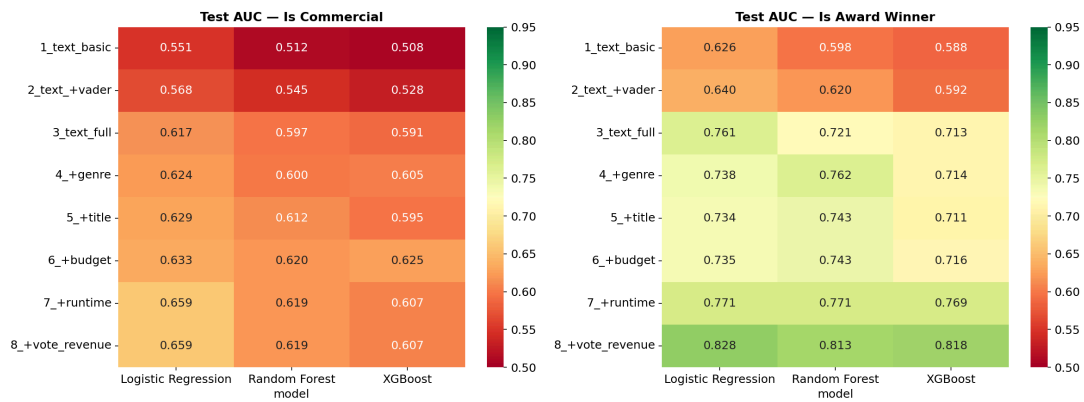
Figures:

- 03_baseline_models_01_progressive_f1_heatmap.png



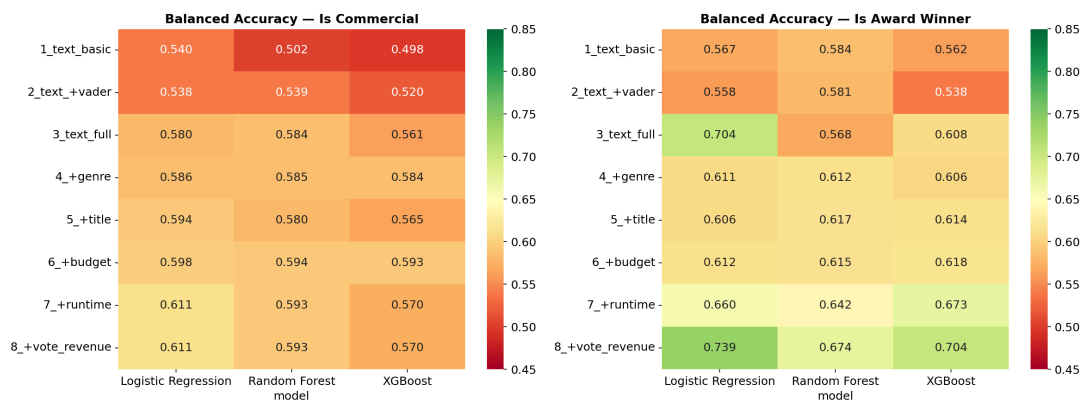
What the figure shows: F1 by feature set (1–8) and model (LR, RF, XGB) for both targets. Results improve from text-only (1–3) to full set (7–8); for commercial F1 around 0.57–0.60, for award lower F1 due to imbalance.

- 03_baseline_models_02_progressive_auc_heatmap.png



What the figure shows: AUC by set and model. AUC for award rises to ~0.77–0.83 on set 8; for commercial around 0.62–0.66. Models rank better than they classify at threshold 0.5.

- 03_baseline_models_03_progressive_bal_acc_heatmap.png



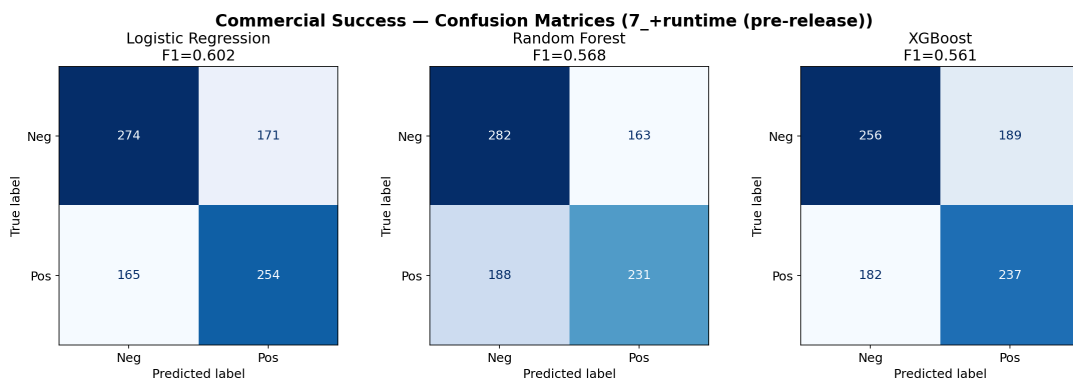
What the figure shows: Balanced accuracy by set and model. Trend similar to F1 and AUC; useful for imbalanced targets.

- 8. Commercial success — confusion matrix, ROC, feature importance**

For chosen set (e.g. 7_+runtime) and target `is_commercial` : confusion matrix on test, ROC curve (all models), feature importance (Random Forest / XGBoost).

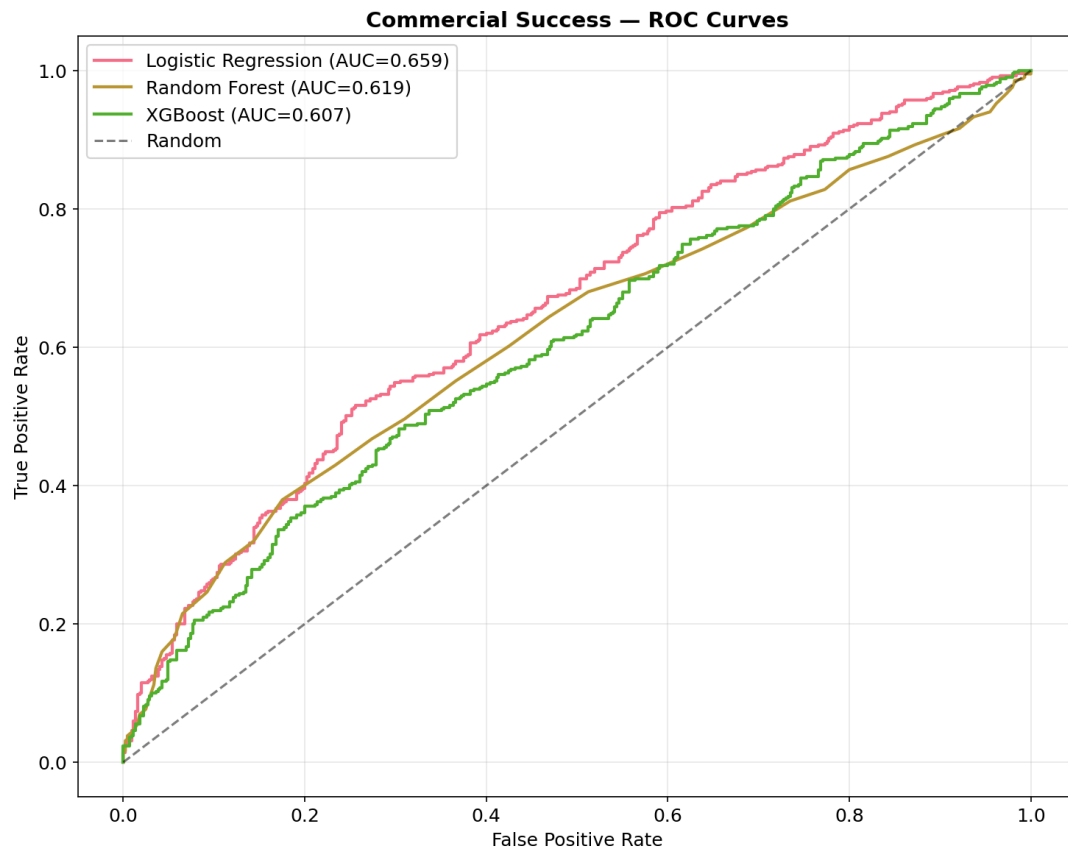
Figures:

- 03_baseline_models_04_commercial_cm.png — confusion matrix (TP, FN, FP, TN).



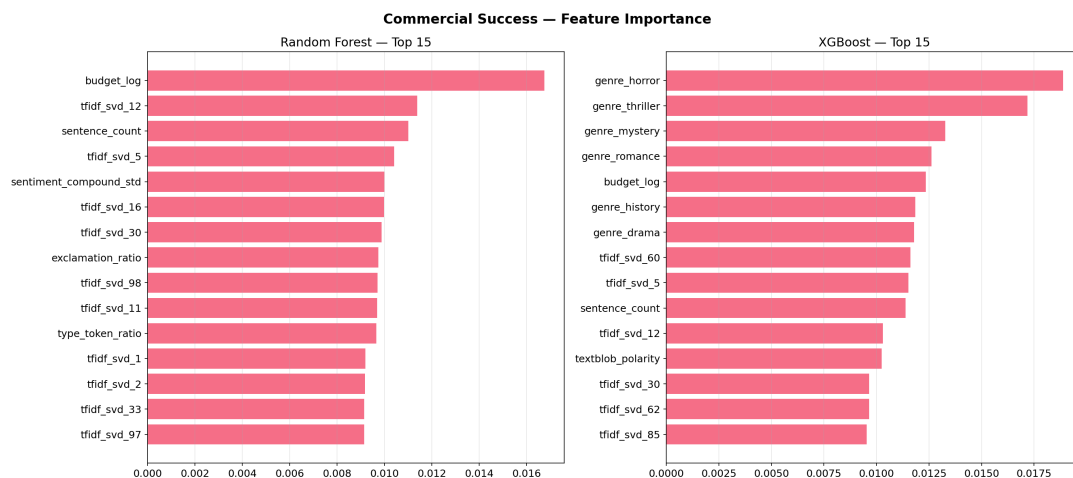
What the figure shows: For commercial success on test: counts of TP, TN, FP, FN. We want larger diagonals (TP, TN); the model is not perfect but makes sense.

- 03_baseline_models_05_commercial_roc.png — ROC curves for LR, RF, XGB.



What the figure shows: ROC curves for all three models; the higher and more to the top-left, the better the AUC. LR typically around 0.66 AUC — solid at distinguishing successful from unsuccessful.

- 03_baseline_models_06_commercial_feat_imp.png — bar plot of feature importance (e.g. vote_count_log, budget_log, runtime, genres).



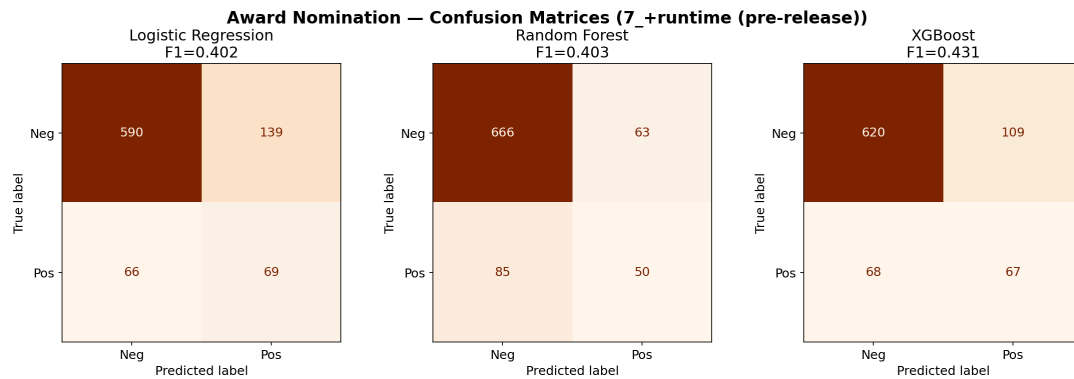
What the figure shows: Which features contribute most (RF/XGB): vote_count_log, budget_log, runtime, and genres. Metadata is a stronger signal than text for this target.

• 9. Award prediction — confusion matrix, ROC, feature importance

Same for is_award_winner (e.g. set 7 or 8).

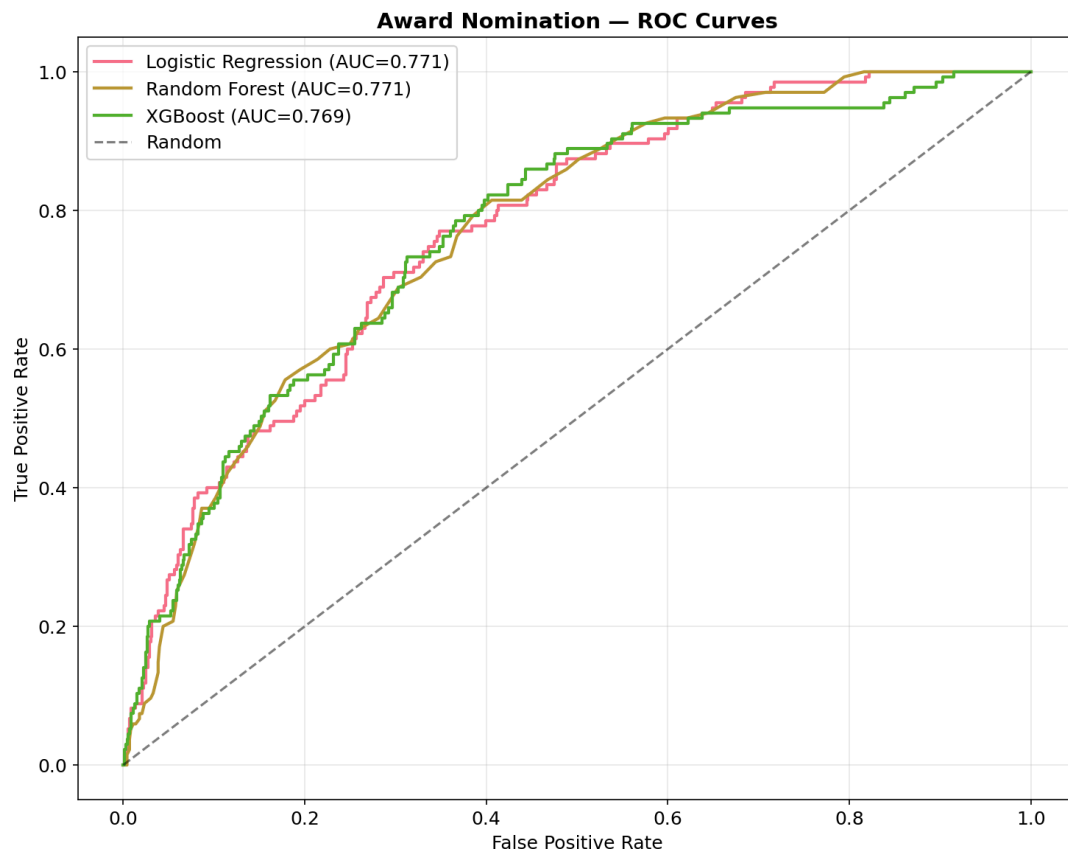
Figures:

- 03_baseline_models_07_award_cm.png



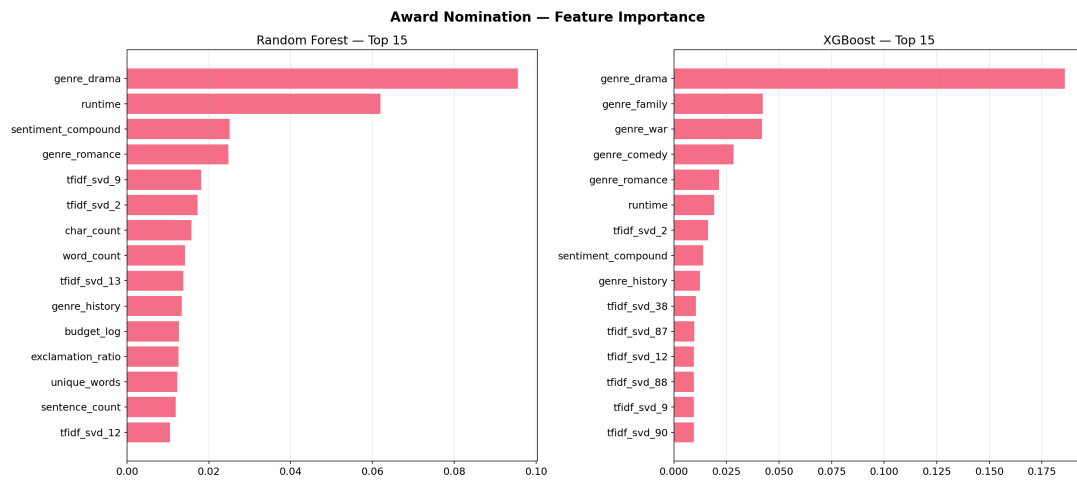
What the figure shows: Confusion matrix for award prediction. The minority class is hard — many false negatives (FN) or false positives (FP). That is why we use F1 at optimal threshold and AUC.

- 03_baseline_models_08_award_roc.png



What the figure shows: ROC curves for award: AUC ~0.77–0.83 on set 8. Models rank movies well by nomination probability; choosing the threshold (e.g. on validation set) improves F1.

- 03_baseline_models_09_award_feat_imp.png — e.g. vote_count_log, runtime, revenue_log, budget_log.

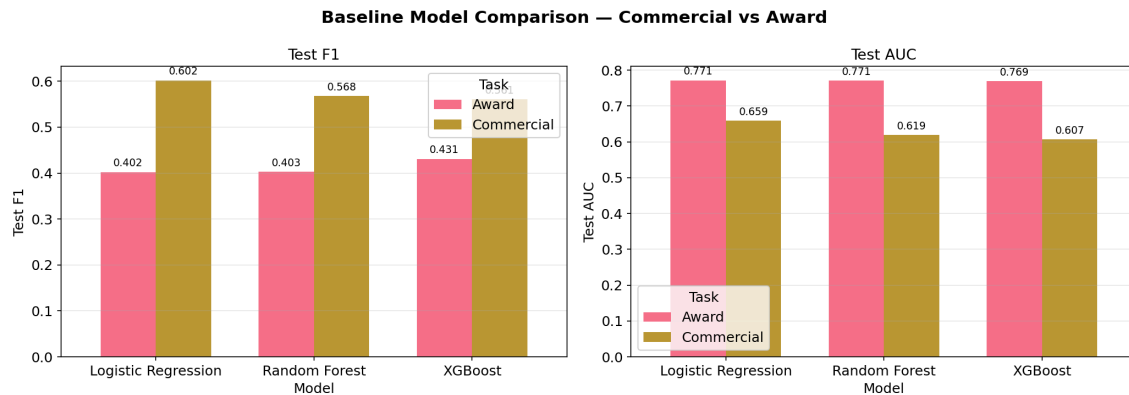


What the figure shows: For award prediction the most important are vote_count_log, runtime, revenue_log, and budget_log. Movies with more votes, longer runtime, and higher revenue are more often nominated.

10. Baseline comparison

Bar plot comparing models (LR, RF, XGB) by F1 or AUC for the chosen set.

Figure: 03_baseline_models_10_baseline_comparison.png



What the figure shows: Direct comparison of LR, RF, and XGB by F1 or AUC for the chosen set. LR often leads slightly for commercial; for award all are similar, with room for improvement via tuning and SVM in KT2.

6.2 What the baseline model figures show

Figure	What it shows
01–03 (heatmaps)	How F1, AUC, and Bal.Acc. improve as features are added (from text to metadata). Set 7 or 8 usually gives the best results.
04 (commercial CM)	For commercial success: counts of correct/incorrect predictions (TP, TN, FP, FN). Shows the model is not perfect but makes sense.

Figure	What it shows
05 (commercial ROC)	ROC curves: the higher and more to the top-left, the better the rank (AUC). LR on set 7 typically around 0.66 AUC.
06 (commercial feat. imp.)	Which features contribute most to predicting commercial success (e.g. vote_count_log, budget_log, runtime).
07–09 (award)	Same for award prediction: confusion matrix, ROC (AUC ~0.77–0.83 for set 8), feature importance (vote_count, runtime, revenue).
10 (comparison)	Simple comparison of the three models on one set (e.g. F1 or AUC).

7. What we use, why, what we expect and what we got

7.1 Techniques and tools

What	Why
Stratified split 70/15/15	So train/val/test keep the same proportion of positives as the full dataset; val for choosing optimal threshold.
SMOTE	To reduce imbalance for awards (minority class ~15%); synthetic minority samples.
class_weight='balanced'	Alternative to SMOTE; class weights inversely proportional to frequency. In practice for award often better than SMOTE (less overfitting).
StandardScaler	Puts all features on the same scale before LR and often before other models.
TruncatedSVD (100)	Reduces TF-IDF dimensionality (5000 columns) to 100 components; eases learning and reduces noise.
F1 and AUC	F1 — balance of precision and recall; AUC — ranking quality. For imbalanced targets we also use F1 at optimal threshold (threshold chosen on validation set).
LR, RF, XGB	LR — linear baseline; RF and XGB — nonlinear, robust; no tuning in KT1.

7.2 Expectations (KT1) and achieved results

- **Commercial success (without revenue in features):**

Expected F1 ~0.60–0.68, AUC ~0.70–0.78 for the set with runtime.

Achieved: F1 ~0.60 (LR on 7_+runtime), F1@opt ~0.65, AUC ~0.66. In line with expectations; LR slightly ahead of RF/XGB.

- **Award:**

Expected F1(opt) ~0.50–0.55, AUC ~0.78–0.87.

Achieved: With set 8_+vote_revenue and SMOTE: F1(opt) ~0.50–0.51 (LR), AUC ~0.83 (LR). With class_weight instead of SMOTE: F1(opt) can be ~0.51; AUC ~0.80. Satisfactory for a baseline.

7.3 Are we satisfied?

Yes, within KT1:

- The pipeline is clear and reproducible; EDA and all figures explain the data and results.
- NLP features are included (from text to TF-IDF SVD and metadata).
- No data leakage for commercial success.
- Baseline results match the specification (F1 and AUC within the stated ranges).

What we would improve in KT2:

- Hyperparameter optimization (RF, XGB, possibly SVM).
- For award: class_weight as default; possibly ADASYN/Borderline-SMOTE.
- Error analysis (which movies are consistently misclassified).
- Ensembles (voting, stacking).

8. Conclusion

This **Phase 1 (KT1) specification** describes **all of Phase 1** of the Movie NLP Success Analyzer project in detail: **how we collected the data** (a pipeline of 10 scripts in `data_pipeline/`), **what we have** (final table, subtitles, NLP features, three notebooks, and shared code), **what each cell does** in notebooks 01, 02, and 03, and **what all figures in figures/ mean**. It also explains **what we use and why** (split, SMOTE, scaling, SVD, metrics), **what we expected and what we got**, and **whether we are satisfied** (yes for KT1; improvements remain for KT2). The specification is written to be understandable to readers without deep prior knowledge of algorithms and NLP techniques, with Mermaid diagrams for the pipeline and Phase 01 structure.